LEVELUP CLOUD TRAININGS

PySpark DataFrame Masterclass

Module 1 — Industry Interview Assignment

Creating DataFrames • Row Class • Primitive Types • Complex Types • Date/Timestamp

Questions	Sections	Difficulty	Style
40	5	Easy→Hard	Interview

Instructions

Each question is tagged with Type (Conceptual / Code Trace / Scenario / Debug / Design), Difficulty, and Company context. Answer as you would in a real technical interview — explain your reasoning clearly, use examples where relevant, and write code where asked. Model answers are provided for self-evaluation.

Section 1: Creating DataFrames

8 Questions | Conceptual · Code Trace · Scenario · Debug

Q1 Conceptual ★ Easy TCS L1 Interview

What are the three ways to create a PySpark DataFrame? Briefly describe each.

Hint: *Think: from existing data (createDataFrame), from files (spark.read), from RDD.*

Model Answer:

1. spark.createDataFrame(data, schema) — from Python collections (lists, tuples, dicts). 2. spark.read.csv/json/parquet(path) — from external files (CSV, JSON, Parquet, HDFS, S3). 3. RDD conversion: rdd.toDF() or spark.createDataFrame(rdd, schema) — from an existing RDD. In production, 80% of DataFrames come from file reads; createDataFrame is mainly used in testing and prototyping.

Q2 Code Trace ★ Easy Wipro Technical Screening

What is the default column name when you run this code? How do you rename it to 'order_id'?

```
data = [101, 102, 103, 104, 105]
df = spark.createDataFrame(data, IntegerType())
df.show()
# Column name = ?
```

Hint: *Spark needs a column name — what does it default to when you pass just a type?*

Model Answer:

Default column name is 'value'. Rename using .toDF('order_id'): spark.createDataFrame(data, IntegerType()).toDF('order_id'). Alternatively use: spark.createDataFrame([(x,) for x in data], ['order_id']) with single-element tuples.

Q3 Scenario ★ Medium Infosys — E-commerce Project

A junior developer at Myntra writes: df = spark.createDataFrame(phone_numbers, IntegerType()). The original data is ['09876543210','08012345678']. What is wrong and how do you fix it?

Hint: *Phone numbers start with 0. What does IntegerType do to leading zeros?*

Model Answer:

Bug: IntegerType drops leading zeros. '09876543210' becomes 9876543210 — the leading 0 is lost, making the number invalid. Fix: Use StringType() instead: spark.createDataFrame(phone_numbers, StringType()).toDF('phone'). Rule: Always store phone numbers, PIN codes, account numbers, and any identifiers that may start with 0 as StringType — NEVER IntegerType.

Q4 Code Trace ★ Easy Cognizant Technical Round

Predict the exact output of printSchema() for this DataFrame. What types does Spark infer?

```
data = [(1,'Amit',29),(2,'Priya',31)]
df = spark.createDataFrame(data, ['id','name','age'])
df.printSchema()  # What does this print?
```

Hint: *Python int → which Spark type? Python str → which Spark type?*

✅Model Answer:

Spark infers: id → LongType (Python int maps to 64-bit Long, NOT IntegerType!), name → StringType, age → LongType. Schema: root |-- id: long (nullable=true) |-- name: string (nullable=true) |-- age: long (nullable=true). Key insight: Python's int is 64-bit by default, so Spark infers LongType. If you need IntegerType (32-bit), pass an explicit schema.

Q5 | **Design** | **★Medium** | HDFC Bank — Data Engineering Team

Design the DataFrame schema for an HDFC Bank transaction table. Fields: transaction_id (large number), account_number (may start with 0), amount (exact money), transaction_date (date only), timestamp (date+time), is_fraud (yes/no flag). Choose appropriate types with justification.

Hint: *Apply the three golden rules: Phone=String, Money=Decimal, Date=DateType.*

✅Model Answer:

transaction_id: LongType (large auto-increment, no leading zeros). account_number: StringType (may have leading zeros, never do math on it). amount: DecimalType(18,2) — NEVER Float/Double for money (rounding errors). transaction_date: DateType — date-only operations (which day, which month). timestamp: TimestampType — exact moment for audit trail and event ordering. is_fraud: BooleanType — binary flag. Schema: StructType([StructField('transaction_id',LongType(),False), StructField('account_number',StringType(),False), StructField('amount',DecimalType(18,2),False), StructField('transaction_date',DateType(),True), StructField('ts',TimestampType(),False), StructField('is_fraud',BooleanType(),False)])

Q6 | **Debug** | **★Medium** | Tech Mahindra — Spark Interview

This code throws an error in production. Identify the bug and fix it.

```
df = spark.createDataFrame(data, schema)
# Process the data...

df.saveAsTextFile("/output/transactions")  # Error!
```

Hint: *What happens when you run saveAsTextFile on a path that already exists? Wrong API?*

✅Model Answer:

Two bugs: (1) saveAsTextFile is an RDD method, NOT a DataFrame method. DataFrames use df.write.csv(path) or df.write.parquet(path). (2) Even for RDD, saveAsTextFile fails if directory already exists. Fix: Use df.write.mode('overwrite').csv('/output/transactions') or add .mode('overwrite') to prevent the directory-exists error. Correct DataFrame write: df.write.mode('overwrite').option('header','true').csv('/output/transactions')

Q7 Scenario ★ Hard Flipkart — Senior Data Engineer Round

You receive a 500GB CSV file with 50 columns. The file has no header row. Spark infers all 50 columns as StringType. Three of them (order_id, amount, order_date) need to be typed correctly. What is the most efficient approach?

 Hint: *Reading twice (once to infer, once to load) is wasteful. Define schema upfront.*

✅Model Answer:

Most efficient: Define schema upfront and pass to spark.read — avoids any type inference scan. For 50 columns where only 3 need specific types, define those 3 explicitly and keep the rest as StringType in the schema, then cast only the 3. Code: schema = StructType([StructField('col1',LongType(),True), StructField('col2',DecimalType(18,2),True), StructField('col3',DateType(),True), ...47 StringType fields...]). Then: df = spark.read.schema(schema).csv('/path/bigfile.csv'). Never use inferSchema=True on files larger than a few MB — it reads the file twice (once to infer, once to load).

Q8 Code Trace ★ Medium HCL — PySpark Technical

What is the output of this code? Why does the column order look unexpected?

```
from pyspark.sql import Row
data=[Row(ID=1,Name='Amit',Age=29),Row(ID=2,Name='Priya',Age=31)]
df=spark.createDataFrame(data)
df.show()  # What order are the columns?
```

 Hint: *When you create a DataFrame from Row objects, how does Spark order the columns?*

✅Model Answer:

Output columns will be in ALPHABETICAL order: Age, ID, Name — not the order you defined them in Row. This is because Row stores named arguments as a sorted dictionary. Output: +---+---+------+ |Age| ID| Name| +---+---+------+ | 29| 1| Amit| To control column order: use toDF('ID','Name','Age') after createDataFrame, or pass an explicit StructType schema with fields in your desired order.

Section 2: The Row Class

8 Questions | Conceptual · Code Trace · Scenario · Design

Q9 Conceptual ★ Easy Accenture L1

What is the Row class in PySpark? How is it different from a Python tuple? When would you use Row instead of a tuple?

Hint: *Named fields vs positional fields. Readability vs brevity.*

Model Answer:

A Python tuple is positional only: row[0], row[1] — no names. A Row object allows access by name: row.Name, row['Name'], or by position: row[0]. Row is more readable and self-documenting. Use Row when: (1) Converting RDDs to DataFrames — Row adds column names. (2) Working with named fields makes code more maintainable. (3) Creating complex/nested data structures. Tuples are fine for quick, small DataFrames where column names are passed separately. Row is preferred in production for clarity.

Q10 Code Trace ★ Easy Wipro / TCS

How many ways can you access the value 'Amit' from this Row object? List all valid syntax options.

```
from pyspark.sql import Row
row1 = Row(ID=1, Name='Amit', Age=29)
# How many ways to get 'Amit'?
```

Hint: *Think: by name, by dict key, by position.*

Model Answer:

Three valid ways: (1) row1.Name — attribute access by field name (most readable). (2) row1['Name'] — dictionary-style key access. (3) row1[1] — positional access. Note: position depends on how Row stores fields. For named Row (Row(ID=1,Name='Amit',Age=29)), Spark may sort alphabetically so row1[1] may return based on sorted order. Always prefer row1.Name for clarity.

Q11 Scenario ★ Medium Infosys BPM

A Python script returns data as a list of dictionaries from a MySQL query cursor. Convert this to a PySpark DataFrame preserving all 4 fields as column names. Write the complete code.

Hint: *Use the ** unpacking operator on each dictionary to create Row objects.*

Model Answer:

Complete solution: from pyspark.sql import Row; data = [{'emp_id':1,'name':'Rahul','dept':'IT','salary':75000}, {'emp_id':2,'name':'Priya','dept':'HR','salary':65000}]; row_data = [Row(**d) for d in data]; df = spark.createDataFrame(row_data); df.show(). The Row(**d) pattern unpacks {'emp_id':1,'name':'Rahul'} into Row(emp_id=1, name='Rahul') automatically. This is the standard pattern for converting dict-based data (API responses, DB cursor results) to DataFrames.

Q12 Code Trace ★ Medium Mphasis Technical Round

Predict what this nested Row code outputs for person.Address.City and person.Address.Zip.

```
from pyspark.sql import Row
address = Row(City='Mumbai', Zip='400001')
person = Row(ID=1, Name='Amit', Address=address)
print(person.Address.City)
print(person.Address.Zip)
```

 Hint: *Nested Row access uses chained dot notation.*

✅Model Answer:

Output: person.Address.City → 'Mumbai'. person.Address.Zip → '400001'. The Address field is itself a Row object, so you access its sub-fields using another dot. This is exactly how StructType columns work in DataFrames — dot notation for nested fields. The DataFrame equivalent is df.select('Address.City').

Q13 Scenario ★ Medium Capgemini — Data Migration Project

An old PySpark job produces an RDD of tuples: (employee_id, employee_name, department, salary). Convert this RDD to a properly named DataFrame using the Row pattern. Show the complete code.

 Hint: *Use rdd.map(lambda x: Row(...)) then spark.createDataFrame().*

✅Model Answer:

Complete code: from pyspark.sql import Row; rdd = spark.sparkContext.parallelize([(1,'Rahul','IT',75000),(2,'Priya','HR',65000),(3,'Arjun','Finance',80000)]); rdd_rows = rdd.map(lambda x: Row(employee_id=x[0], employee_name=x[1], department=x[2], salary=x[3])); df = spark.createDataFrame(rdd_rows); df.show(); df.printSchema(). This is the classic RDD→DataFrame migration pattern used in every legacy Spark upgrade project.

Q14 Conceptual ★ Easy Cognizant Entry-Level

What does dynamic Row creation mean in PySpark? Give an example using a Row template.

 Hint: *Creating a Row 'class template' first, then using it to instantiate records.*

✅Model Answer:

Dynamic Row creation: First define a Row template with field NAMES only (no values), then use it to create instances. Example: Person = Row('name','age','city') — this is the template. person1 = Person('Rahul',28,'Hyderabad') — this is an instance. person2 = Person('Priya',32,'Mumbai'). This pattern is useful when field names are fixed but values vary. It's like defining a named tuple class. You can then create a list of these and pass to createDataFrame.

Q15 Debug ★ Hard Razorpay — Data Engineering

This code tries to create a DataFrame from a list of dicts but throws an AttributeError. Fix it.

```
data = [{'txn_id':1,'amount':500.0},{'txn_id':2,'amount':1200.0}]
df = spark.createDataFrame(data)  # AttributeError!
df.show()
```

Hint: *createDataFrame() doesn't directly accept dicts — what intermediate step is needed?*

✅Model Answer:

Bug: spark.createDataFrame() cannot directly accept a list of plain dicts. It needs either tuples, Rows, or an explicit schema. Fix using Row unpacking: row_data = [Row(**d) for d in data]; df = spark.createDataFrame(row_data). Alternative fix using pandas (for small data): import pandas as pd; df = spark.createDataFrame(pd.DataFrame(data)). Production recommendation: Row(**d) pattern is native PySpark — prefer it over pandas conversion for large datasets.

Q16 | **Design** | **★ Hard** | Amazon India — Data Engineering

Design a nested DataFrame schema for an Amazon order. Each order has: order_id, customer_id, a list of ordered items (each item has product_id, quantity, unit_price), delivery_address (street, city, pincode), and order_timestamp. Use Row objects to create sample data and define the complete schema.

Hint: *Use ArrayType for the items list, StructType for the nested item and address.*

✅Model Answer:

Schema design: order_id: LongType, customer_id: LongType, items: ArrayType(StructType([StructField('product_id',LongType()), StructField('quantity',IntegerType()), StructField('unit_price',DecimalType(10,2))])), delivery_address: StructType([StructField('street',StringType()), StructField('city',StringType()), StructField('pincode',StringType())]), order_timestamp: TimestampType. Sample Row: Row(order_id=1001, customer_id=5, items=[Row(product_id=200,quantity=2,unit_price=Decimal('299.99'))], delivery_address=Row(street='MG Road',city='Bangalore',pincode='560001'), order_timestamp=datetime(2024,1,15,10,30)). This nested structure is standard for e-commerce order data in data lakes.

Section 3: Primitive Data Types

8 Questions | Conceptual · Scenario · Code · Design

Q17 Conceptual ★Easy TCS / Accenture

What is the difference between IntegerType and LongType in PySpark? When should you use each?

Hint: *Size difference: 32-bit vs 64-bit. Think about value ranges.*

✓Model Answer:

IntegerType: 32-bit signed, range -2.1 billion to 2.1 billion. LongType: 64-bit signed, range -9.2 quintillion to 9.2 quintillion. Use IntegerType for: age, item count, small IDs (< 2 billion). Use LongType for: large auto-increment IDs (Flipkart has 10B+ orders), Unix timestamps in milliseconds, large financial counts. Important: Python's default int maps to LongType in Spark — when Spark infers from Python data, it always creates LongType, never IntegerType.

Q18 Scenario ★Medium ICICI Bank — Fintech Interview

ICICI Bank stores transaction amounts using DoubleType. A senior engineer says this is wrong. Why? What should they use instead? Give a concrete numeric example showing the problem.

Hint: *Floating-point precision: 0.1 + 0.2 in binary. Try it in Python.*

✓Model Answer:

Problem: Floating-point numbers cannot represent all decimal fractions exactly. In Python (same issue in Spark): 0.1 + 0.2 = 0.30000000000000004 (not exactly 0.30). For a ₹10.10 + ₹20.20 transaction, DoubleType gives ₹30.299999999... instead of ₹30.30 — rounding error in bank calculations. Fix: Use DecimalType(18,2) — exact decimal arithmetic. DecimalType(18,2) means: up to 18 total digits, 2 after decimal point. ₹10.10 + ₹20.20 = exactly ₹30.30. This is why all financial systems (SWIFT, core banking) use Decimal, never Float/Double.

Q19 Code Trace ★Medium Infosys Technical Round

What type does Spark infer for each column when no schema is provided? What are the three 'wrong type' issues in this data?

```
data=[(1,'Amit','09876543210',55000.50),(2,'Priya','08012345678',72000.00)]
df=spark.createDataFrame(data,['emp_id','name','phone','salary'])
df.printSchema()  # What types are inferred? What are the problems?
```

Hint: *What types does Python → Spark inference produce? What edge cases are lost?*

✓Model Answer:

Inferred types: emp_id → LongType (Python int), name → StringType, phone → StringType (correct! Spark is smart enough for this), salary → DoubleType (Python float). Three issues: (1) emp_id should be IntegerType (not Long) if IDs < 2 billion — wastes 4 bytes per row at scale. (2) salary should be DecimalType(12,2) for exact financial calculations, not DoubleType. (3) If phone were a number (like

9876543210 without quotes), Spark would infer LongType and a phone number starting with 0 would lose it. Always define explicit schema.

Q20 Conceptual ★ Easy Cognizant / Wipro

When should you use FloatType vs DoubleType? Can you use either for financial calculations?

 Hint: *Precision difference. Financial = exact vs scientific = approximate.*

✅Model Answer:

FloatType: 32-bit, ~7 decimal digit precision — suitable for approximate measurements (sensor readings, ratings like 4.3 stars, GPS coordinates with 5-digit precision). DoubleType: 64-bit, ~15 decimal digit precision — suitable for scientific calculations, statistical models, coordinates. NEITHER should be used for financial calculations — both have floating-point rounding errors. For money, always use DecimalType(precision, scale). Rule: Float for approximate science, Double for higher-precision science, Decimal for exact business values.

Q21 Scenario ★ Hard Navi Technologies

Navi processes 5 billion loan EMI records per month. Each record has: loan_id (10-digit), emi_amount (money), outstanding_balance (money), emi_date (date), processing_timestamp (with time), is_paid (yes/no), bounce_count (max 10). Choose the optimal type for each field, minimizing storage while maintaining accuracy.

 Hint: *Storage optimization: smallest correct type for each field. Think about ranges.*

✅Model Answer:

Optimized schema: loan_id: LongType (10-digit number > 2.1B max for IntegerType). emi_amount: DecimalType(12,2) — exact money, up to ₹9,999,999,999.99. outstanding_balance: DecimalType(14,2) — slightly larger range for total balance. emi_date: DateType — date operations, no time needed. processing_timestamp: TimestampType — exact moment needed. is_paid: BooleanType — 1 bit, binary flag. bounce_count: ByteType — max 10, well within -128 to 127. Storage saving: Using ByteType for bounce_count instead of IntegerType saves 3 bytes per record. At 5B records: 15GB storage saved just from that one field!

Q22 Code Trace ★ Medium Deloitte — PySpark Round

What happens when you try to store the value 200 in a ByteType column? What is the correct fix?

```
schema=StructType([StructField('order_id',IntegerType()),StructField('item_count',ByteType())])
data=[(1001,200),(1002,3),(1003,150)]
df=spark.createDataFrame(data,schema)
# What happens to item_count=200?
```

 Hint: *ByteType range is -128 to 127. What happens when you exceed it?*

✅Model Answer:

Result: Spark throws an AnalysisException or data is silently corrupted (out-of-range values become null). 200 exceeds ByteType max of 127. Fix: Use ShortType (max 32,767) for values up to 32K, or IntegerType for larger values. Code fix: StructField('item_count', ShortType(), True) — ShortType handles 200 easily with range up to 32,767. Lesson: Always check value ranges before choosing a type. Common traps: ByteType for 'small numbers' when value can reach 200+, IntegerType for large IDs that exceed 2.1 billion.

Q23 | **Conceptual** | **★ Easy** | TCS Digital Interview

Why is BooleanType better than IntegerType for storing a flag column (like 'is_active', 'is_verified')?

Hint: *Storage, readability, and semantic clarity.*

✅Model Answer:

Three reasons: (1) Storage: BooleanType uses 1 bit; IntegerType uses 32 bits. For 1 billion user records, that's 125MB vs 4GB just for one flag column. (2) Readability: df.filter(col('is_active')==True) is clearer than df.filter(col('is_active')==1). (3) Type safety: Boolean columns can only be True or False — prevents invalid values like is_active=5. SQL: WHERE is_active = true reads naturally. Additionally, Spark's Catalyst optimizer can use BooleanType for predicate pushdown more efficiently.

Q24 | **Scenario** | **★ Hard** | Google India — Data Engineer

You are processing Wikipedia article data. Each article has: article_id (10 billion+), title (any Unicode text including Hindi, Tamil, Arabic), content (up to 10MB text), view_count (in billions), last_edited (date + time), is_featured (yes/no), revision_number (up to 2 million). Design the schema. Which field presents the biggest challenge and why?

Hint: *Consider: LongType limit, StringType for Unicode, content storage in Spark.*

✅Model Answer:

Schema: article_id: LongType (10B > Integer max, Long handles up to 9.2Q). title: StringType (handles all Unicode — Hindi, Arabic etc). content: StringType (handles large text — Spark doesn't have a 'TEXT' type, StringType is variable-length). view_count: LongType (billions exceed Integer). last_edited: TimestampType (need exact edit time). is_featured: BooleanType. revision_number: IntegerType (2M << 2.1B limit, Int is fine). Biggest challenge: content field. Each article up to 10MB as StringType is fine for schema, but Spark keeps data in executor memory. A partition with 1000 articles = 10GB per partition — likely OOM. Solution: Store content in external storage (S3/GCS), keep only article_id + metadata in Spark, fetch content on demand.

Section 4: Complex Data Types (Array, Map, Struct)

8 Questions | Conceptual · Code · Scenario · Design

Q25 Conceptual ★ Easy TCS / HCL L1

Explain ArrayType in PySpark. When would you use it instead of creating separate columns?

Hint: *What is stored in ArrayType? When does one-column-per-value not scale?*

Model Answer:

ArrayType(elementType) stores a list of values of the same type in a single column. Use ArrayType when: (1) A record can have a variable number of values of the same type — a user can have 0 to N hobbies, an order can have 1 to M items. Creating separate hobby_1, hobby_2, hobby_3 columns doesn't scale (what if someone has 20 hobbies?). (2) The values are semantically a collection belonging to one entity — tags on a blog post, phone numbers for a contact. (3) You need to use array functions: explode (one row per element), size (count elements), array_contains (check membership).

Q26 Code Trace ★ Medium Infosys — Spark Technical

What does explode() do to this DataFrame? How many rows does the output have? Show the exa

```
from pyspark.sql.functions import explode
data=[(1,'Amit',['Reading','Traveling','Music']),(2,'Priya',['Cooking','Yoga']),(3,'Vikram',['Gami
schema='id int, name string, hobbies array<string>'
df=spark.createDataFrame(data,schema)
df.select('name',explode('hobbies').alias('hobby')).show()
# How many rows in output?
```

Hint: *explode() creates one row per element in the array. Count total elements across all rows.*

Model Answer:

explode('Hobbies') creates one row per hobby per person: Amit → 3 rows (Reading, Traveling, Music), Priya → 2 rows (C (Gaming, Hiking, Photography). Total output: 8 rows (3+2+3). Output: +------+-----------+ |Name |hobby | +------+-------- |Traveling | |Amit |Music | |Priya |Cooking | |Priya |Yoga | |Vikram|Gaming | ...

Q27 Conceptual ★ Easy Cognizant / Mphasis

What is MapType in PySpark? How do you define it? Give a real-world example where MapType is more appropriate than separate columns.

Hint: *MapType stores key-value pairs. When are keys variable/unknown upfront?*

Model Answer:

MapType(keyType, valueType) stores a dictionary of key-value pairs in one column. Real-world example: Student exam scores — {'Math':85, 'Science':90, 'English':78}. The problem with separate columns: different students take different subjects. You can't know all possible subjects upfront, and they vary per student. With MapType: one 'scores' column stores whatever subjects each student has. Access: df.select(col('scores')['Math']) gets Math score. Other examples:

product_attributes{'color':'red','size':'XL','material':'cotton'},
user_settings{'theme':'dark','language':'Hindi','notifications':'off'}.

Q28 Scenario ★ Hard Swiggy — Data Engineering

Swiggy's order table has a 'cart_items' column of MapType(StringType, IntegerType) where keys are dish names and values are quantities. Write the code to: (1) Find orders where 'Biryani' was ordered, (2) Get the quantity of Biryani in each order.

 Hint: *Use array_contains for check, col()['key'] for value access in MapType.*

✅Model Answer:

Solution: from pyspark.sql.functions import col, map_keys; # Check if Biryani was ordered: df.filter(array_contains(map_keys('cart_items'), 'Biryani')).show(); # Get Biryani quantity: df.select('order_id', col('cart_items')['Biryani'].alias('biryani_qty')).show(); # Combined: df.filter(col('cart_items')['Biryani'].isNotNull()).select('order_id', col('cart_items')['Biryani'].alias('qty')).show(). Note: col('cart_items')['Biryani'] returns null for orders that don't have Biryani — use isNotNull() to filter those out.

Q29 Code Trace ★ Medium Wipro — Spark Assessment

What does printSchema() show for this nested StructType DataFrame? Draw the schema tree.

```
schema=StructType([StructField('ID',IntegerType()),StructField('Name',StringType()),StructField('A
df=spark.createDataFrame([(1,'Amit',('MG Road','Mumbai','400001'))],schema)
df.printSchema()  # Draw the output tree
```

 Hint: *StructType creates a 'struct' in the schema. Nested structs show with indentation.*

✅Model Answer:

printSchema() output: root |-- ID: integer (nullable=true) |-- Name: string (nullable=true) |-- Address: struct (nullable=true) df.select('Name','Address.City').show(). Access nested using col(): df.select(col('Address.City'),col('Address.ZipCode')).sl

Q30 Design ★ Hard Flipkart — System Design Round

Design a complete PySpark DataFrame schema for Flipkart's product catalog. Each product has: product_id, name, category, brand, price, tags (multiple), specifications (key-value pairs like color→red, RAM→8GB), seller_info (seller_id, seller_name, rating), images (list of URLs). Choose the right type for each field.

 Hint: *Identify which fields are: scalar, list, map, or nested record.*

✅Model Answer:

Schema: product_id: LongType (large unique IDs). name: StringType. category: StringType. brand: StringType. price: DecimalType(10,2) — exact money. tags: ArrayType(StringType) — ['electronics','sale','featured']. specifications: MapType(StringType,StringType) — {'color':'Midnight Black','RAM':'8GB','Storage':'256GB'}. seller_info: StructType([StructField('seller_id',LongType()), StructField('seller_name',StringType()), StructField('rating',FloatType())]) — nested record (approx rating is OK for display). images: ArrayType(StringType) — list of S3 URLs. This mixed-type schema

covers flat scalars + arrays + maps + nested struct — exactly what real product catalog data looks like.

Q31 Scenario ★ Medium Paytm — Data Analytics

A Paytm analyst queries: df.select(col('metadata')['amount']).show() on a MapType column, but all rows show null. The data definitely has 'amount' in the map. What is the likely bug?

 Hint: *MapType is case-sensitive. What might differ between the data and the query?*

✅Model Answer:

Most likely bug: Key name case mismatch. MapType keys are case-sensitive. If the data stores 'Amount' (capital A) but the query uses 'amount' (lowercase), every lookup returns null. Fix: Verify exact key names with df.select(map_keys('metadata')).show() first to see actual key names. Other possible bugs: (1) Key has trailing space — 'amount ' vs 'amount'. (2) Key is actually stored as integer type but query uses string. Always inspect actual keys with map_keys() before querying MapType columns.

Q32 Design ★ Hard Zepto — Real-Time Data Platform

Zepto's order stream has complex nested data: each order_event has order_id, event_type (PLACED/CONFIRMED/DISPATCHED/DELIVERED), rider (rider_id, current_lat, current_lng, name), items ordered (array of item_id+quantity+price), and event_timestamp. Design the full PySpark schema and write sample Row data for one event.

 Hint: *Combine all complex types: StructType for nested, ArrayType for items list.*

✅Model Answer:

Schema: StructType([StructField('order_id',LongType()), StructField('event_type',StringType()), StructField('rider',StructType([StructField('rider_id',LongType()), StructField('name',StringType()), StructField('lat',DoubleType()), StructField('lng',DoubleType())])), StructField('items',ArrayType(StructType([StructField('item_id',LongType()), StructField('qty',IntegerType()), StructField('price',DecimalType(8,2))]))), StructField('event_ts',TimestampType())]). Sample Row: Row(order_id=90001, event_type='DISPATCHED', rider=Row(rider_id=5001,name='Ravi',lat=17.4239,lng=78.4738), items=[Row(item_id=101,qty=2,price=Decimal('49.00')),Row(item_id=205,qty=1,price=Decimal('35.50'))], event_ts=datetime(2024,1,15,10,5,30))

Section 5: Date, Timestamp & Type Casting

8 Questions | Conceptual · Code · Scenario · Debug

Q33 Conceptual ★ Easy TCS / Wipro L1

What is the difference between DateType and TimestampType in PySpark? Give one real example where you would use each.

Hint: *Date = day only. Timestamp = day + time. What operations does each enable?*

Model Answer:

DateType: Stores only the calendar date (year, month, day) — no time component. Example: customer date_of_birth, product launch_date, policy_expiry_date. Use when time doesn't matter. Supports: year(), month(), day(), date_add(), datediff(). TimestampType: Stores date AND time (up to microsecond). Example: payment_timestamp, login_event_time, order_placed_at. Use when exact moment matters — for audit trails, SLA calculation, event ordering. Supports all DateType operations PLUS hour(), minute(), second(), unix_timestamp().

Q34 Code Trace ★ Medium Infosys — Date Processing

What does this code output? Which row(s) return a valid result and which return null?

```
from pyspark.sql.functions import to_date
data=[('2024-01-15',),('15/01/2024',),('2024-13-01',)]
df=spark.createDataFrame(data,['date_str'])
df.withColumn('parsed_date',to_date('date_str','yyyy-MM-dd')).show()
# Which rows are null?
```

Hint: *to_date() returns null if the string doesn't match the specified format.*

Model Answer:

Output: row 1 (2024-01-15) → DateType correctly parsed. row 2 (15/01/2024) → null (format is yyyy-MM-dd but data is dd/MM/yyyy — mismatch!). row 3 (2024-13-01) → null (month 13 doesn't exist — invalid date). Best practice: Always validate date formats before loading. Use try_to_date() (Spark 3.2+) which returns null instead of erroring, then filter nulls to find bad rows. In production, log null count after date conversion: invalid_dates = df.filter(col('date_col').isNull()).count().

Q35 Scenario ★ Medium CRED — Analytics Team

CRED needs to calculate: (1) age of each loan in days from disbursement_date to today, (2) whether a loan is more than 90 days old (boolean), (3) year and month of disbursement. Write the complete PySpark code using date functions.

Hint: *Use datediff(), current_date(), year(), month() from pyspark.sql.functions.*

Model Answer:

from pyspark.sql.functions import datediff, current_date, year, month, col; df.select('loan_id', 'disbursement_date', datediff(current_date(),'disbursement_date').alias('age_days'), (datediff(current_date(),'disbursement_date') > 90).alias('is_old_loan'),

year('disbursement_date').alias('disburse_year'), month('disbursement_date').alias('disburse_month')).show(). Note: datediff(end_date, start_date) — order matters. datediff(today, past_date) gives positive days elapsed. datediff(past_date, today) gives negative.

Q36 | Debug | ★ Medium | Accenture — PySpark Debug Round

This date calculation gives wrong results — the 'days_since_order' column shows negative numbers for recent orders. What is the bug?

```
from pyspark.sql.functions import datediff,current_date
df=df.withColumn('days_since_order',datediff('order_date',current_date()))
df.show()
# Shows negative values for recent orders — why?
```

Hint: *datediff(end, start) vs datediff(start, end) — order of arguments matters.*

✓Model Answer:

Bug: The arguments to datediff() are in wrong order. datediff(col1, col2) = col1 - col2. The code has datediff('order_date', current_date()) = past_date - today = NEGATIVE number. Fix: Swap the arguments: datediff(current_date(), 'order_date') = today - past_date = POSITIVE days elapsed. Common mistake: developers read it as 'diff from A to B' but it's actually 'A minus B'. Always verify the sign of your result with known test data before deploying.

Q37 | Scenario | ★ Hard | Groww — Data Pipeline Interview

Groww stores stock trade timestamps as Unix epoch milliseconds (long integer, e.g., 1705297245000). Convert this to a proper TimestampType column and extract the trade hour and minute. Write the complete code.

Hint: *Unix ms to Timestamp: divide by 1000 (to get seconds), then use from_unixtime() or cast.*

✓Model Answer:

from pyspark.sql.functions import col, from_unixtime, hour, minute, to_timestamp; # Method: divide ms by 1000 to get seconds epoch, then convert: df.withColumn('trade_ts', from_unixtime(col('epoch_ms')/1000).cast(TimestampType())).withColumn('trade_hour', hour('trade_ts')).withColumn('trade_minute', minute('trade_ts')).show(). Alternative: col('epoch_ms').divide(1000).cast(TimestampType()). Key insight: Unix epoch is in seconds, but many systems (JavaScript, Kafka) use milliseconds — always check and divide by 1000 before conversion. Off by 1000 = timestamps in year 1970 instead of today!

Q38 | Code Trace | ★ Easy | Capgemini Technical

After reading a CSV, all columns are StringType. Show the cast() code to convert 'age' to IntegerT DecimalType(10,2). Also show the shorthand string version of cast.

Hint: *withColumn + cast(). Show both class and string versions.*

✓Model Answer:

Full code: from pyspark.sql.functions import col; from pyspark.sql.types import IntegerType, DecimalType; # Using type c df.withColumn('age', col('age').cast(IntegerType())).withColumn('salary', col('salary').cast(DecimalType(10,2))); # Using s

(equivalent): df = df.withColumn('age', col('age').cast('int')).withColumn('salary', col('salary').cast('decimal(10,2)')); # Chai
df.withColumn('age',col('age').cast('int')).withColumn('salary',col('salary').cast('decimal(10,2)')).withColumn('join_date',to_
MM-dd'))

Q39 | Scenario | ★ Hard | PhonePe — Senior Data Engineer

PhonePe's raw transaction log has timestamps as strings in mixed formats: some rows have '2024-01-15 10:30:45', others have '15-Jan-2024 10:30', and some have Unix epoch strings ('1705297245'). Design a robust parsing strategy that handles all three formats without failing.

Hint: *coalesce() on multiple to_timestamp() attempts — first non-null wins.*

✅Model Answer:

Strategy: Try parsing with each format, use coalesce() (SQL function, not the partition one) to pick first non-null result. Code: from pyspark.sql.functions import to_timestamp, coalesce, col, from_unixtime; df.withColumn('ts', coalesce(to_timestamp('raw_ts','yyyy-MM-dd HH:mm:ss'), to_timestamp('raw_ts','dd-MMM-yyyy HH:mm'), from_unixtime(col('raw_ts').cast('long')).cast(TimestampType()))).show(). coalesce() here is the SQL function (not the partition function!) — it returns the first non-null value from its arguments. Rows with unrecognized format get null — flag these for data quality monitoring.

Q40 | Design | ★ Hard | Juspay — Data Platform Architecture

Design an end-to-end schema + loading pipeline for Juspay's payment event data. Raw data arrives as JSON with inconsistent types (amount sometimes string, sometimes number), timestamps in mixed formats, and optional nested fields. Walk through: schema definition, safe loading, type correction, and validation.

Hint: *JSON loading, schema enforcement, type casting, null handling — complete pipeline.*

✅Model Answer:

Complete pipeline: Step 1 — Define strict schema: StructType with all expected fields, using nullable=True for optional fields. Step 2 — Load JSON with schema enforcement: df = spark.read.schema(strict_schema).json(path). Spark uses null for missing/wrong-type fields instead of erroring. Step 3 — Fix amount (could be string or number): df.withColumn('amount', coalesce(col('amount_num').cast(DecimalType(18,2)), col('amount_str').cast(DecimalType(18,2)))). Step 4 — Fix timestamps: withColumn('ts', coalesce(to_timestamp formats)). Step 5 — Validate: bad_rows = df.filter(col('amount').isNull() | col('ts').isNull()).count(). Step 6 — Log bad_rows count to monitoring system via accumulator or metrics. Step 7 — Filter good rows for downstream processing, write bad rows to dead-letter path for manual inspection.